Integrating and Benchmarking KpqC in TLS/X.509

Minjoo Sim[1[0000−0001−5242−214X]], Gyeongju Song[1[0000−0003−0069−9061]], Minwoo Lee[1[0000−0002−2356−3055]], Seyoung Yoon[1], Anubhab Baksi[2[0000−0002−5639−7372]], and Hwajeong Seo[1[0000−0003−0069−9061]]

[1]Hansung University, South Korea,
[2] Nanyang Technological University, Singapore,
{minjoos9797, thdrudwn98, minunejip, sebbang99, hwajeong84}@gmail.com
anubhab.baksi@ntu.edu.sg

Abstract. This paper reports on the implementation and performance evaluation of Korean Post-Quantum Cryptography standards within existing TLS/X.509 infrastructure. We integrated HAETAE, AIMer, SMAUG-T, and NTRU+—the four KpqC standard algorithms—into the OpenSSL ecosystem via a modified liboqs framework. Then, we measured static overhead (certificate size) and dynamic overhead (TLS handshake latency) under both computational-bound (localhost) and network-bound (LAN) settings. Our results indicate that, focusing on the Korean standards, KpqC certificates are 11.5–48 times larger than the classical ECC baseline. In performance, the tested KpqC KEMs increase handshake latency by over 750% in computation-bound tests (localhost) and by up to 35% in network-bound tests (LAN). To our knowledge, this study constitutes the first practical evaluation of KpqC standards in real-world TLS environments, providing concrete performance data to guide post-quantum migration strategies.

Keywords: Post-quantum Cryptography · KpqC Algorithms · TLS/X.509 Integration

1 Introduction

Quantum computers pose significant threats to classical public-key schemes such as RSA and elliptic-curve cryptography (ECC) [1]. In June 2022, NIST selected four post-quantum algorithms—CRYSTALS-KYBER [2], CRYSTALS-DILITHIUM [3], FALCON [4], and SPHINCS+ [5]—as standards for the post-quantum era. Integrating these into X.509-based Public Key Infrastructure (PKI) is essential to secure TLS communications against quantum attacks [6–9], yet post-quantum schemes incur significant overheads, including larger certificate sizes and longer TLS handshake times.

While extensive empirical studies exist for NIST candidates, no practical evaluation of Korean Post-Quantum Cryptography (KpqC) standards has been performed in real-world TLS environments. This lack of data makes it difficult

for organizations to decide whether and how to adopt KpqC. To bridge this gap, we present a unified analysis of both static (certificate size) and dynamic (handshake performance) overheads incurred by the PQC transition. We argue that these costs must be evaluated jointly, since the choice of signature scheme directly affects certificate size while the key-exchange scheme determines runtime latency—a critical trade-off for system architects. Our study is enabled by a novel testbed in the Open Quantum Safe (OQS) framework, in which we ported KpqC signature algorithms (HAETAE, AIMer) and KEMs (SMAUG-T, NTRU+) to allow direct, apples-to-apples comparisons. Our experiments confirm that integrating post-quantum schemes into TLS X.509 incurs notable overhead: across all tested algorithms (NIST and KpqC), certificate sizes increase by a factor of 4.6× to 48.8×. The dynamic overhead is most pronounced for KpqC's KEMs, which increase handshake latency by up to 780% in localhost and 38% over LAN. These numbers—drawn from our comprehensive evaluation in Section 4—underscore the trade-offs system architects must consider for post-quantum migration.

The remainder of this paper is organized as follows. Section 2 reviews background, Section 3 describes our integration and experimental setup, Section 4 presents results and analysis, and Section 5 concludes with future research directions.

1.1 Contributions

Our main contributions are threefold:

- **First Implementation of KpqC in a Production-Ready Environment:** We present the first successful integration of four major KpqC algorithms into the widely-used OpenSSL library via the Open Quantum Safe (OQS) framework. This implementation removes a significant technical barrier, enabling immediate research and pilot deployments.
- **Comprehensive Evaluation of Hybrid X.509 Certificates:** We conduct the first systematic benchmark of KpqC-based hybrid certificates, measuring both static (certificate size) and dynamic (TLS 1.3 handshake latency) overheads. Our analysis provides a direct and fair comparison against classical ECC and standardized NIST PQC algorithms.
- **Practical Deployment Insights and Guidelines:** We offer concrete deployment guidance based on performance characterization in both computational bound (localhost) and network-bound (LAN) scenarios. Our findings include counter-intuitive results, such as the mitigation of high computational costs in networked environments, providing a new evidence-based perspective for system architects planning PQC migration.

2 Background

2.1 Open Quantum Safe & liboqs

The Open Quantum Safe (OQS) project aims to facilitate the transition to post-quantum cryptography (PQC) by integrating quantum-resistant algorithms

Table 1: Parameter Sizes of NIST PQC Standard Algorithms

Type	Algorithm	Security Lvl.	Public Key (Bytes)	Secret Key (Bytes)	Ciphertext/Sig. (Bytes)
KEM	ML-KEM-512	1	800	1,632	768
	ML-KEM-768	3	1,184	2,400	1,088
	ML-KEM-1024	5	1,568	3,168	1,568
Signature	ML-DSA-44	2	1,312	2,528	2,420
	ML-DSA-65	3	1,952	4,000	3,293
	ML-DSA-87	5	2,592	4,864	4,595
Signature	Falcon-512	1	897	1,281	666
	Falcon-1024	5	1,793	2,305	1,280
Signature	SPHINCS+-128s	1	32	64	7,856
	SPHINCS+-128f	1	32	64	17,088
	SPHINCS+-192s	3	48	96	16,224
	SPHINCS+-192f	3	48	96	35,664
	SPHINCS+-256s	5	64	128	29,792
	SPHINCS+-256f	5	64	128	49,856

into existing protocols and applications [10]. Its core library, `liboqs`, provides a standardized C API for a wide range of PQC Key Encapsulation Mechanisms (KEMs) and digital signature schemes, while the associated `oqs-provider` enables their use within the OpenSSL 3.x framework for testing and benchmarking [11]. However, at the time of this research, the OQS ecosystem lacked official support for the candidates in the KpqC competition. This absence presented a significant barrier to their empirical evaluation and hindered a direct comparison against the finalized NIST standards, motivating the implementation work presented in this paper.

2.2 NIST PQC Standard Algorithms

Table 1 summarizes the parameters (security level, public key, private key, ciphertext/signature size) of the major KEM/DSA algorithms standardized by NIST. It provides a baseline for the KpqC performance comparison in this paper, and is also referenced in the comparative discussion in Section 3.

2.3 X.509 Certificates & PQ Hybrid Certification

X.509 certificates, standardized by ITU-T as part of the X.500 series, define a DER-encoded format for public-key data, issuer/subject names, validity periods, and digital signatures [12]. Extensions allow embedding custom fields—used here to carry KpqC public keys and signatures. PQ Hybrid Certification combines classical and post-quantum algorithms within a single certificate to en-

sure continuity and security during migration [13–15]. Common approaches include composite (parallel algorithms), hybrid (dual-algorithm signatures), and chameleon (dynamic field adaptation) certificates [16]. By integrating multiple schemes, composite certificates remain secure if one algorithm is broken, offering adaptable security profiles across environments.

2.4 KPQC Competition

In parallel with global standardization efforts, Korea launched its national post-quantum cryptography competition (KpqC) in 2022 to establish domestic PQC standards [17]. Initially comprising 16 candidates, the competition advanced eight algorithms to Round 2 in December 2023: four Key Encapsulation Mechanisms (NTRU+, PALOMA, REDOG, SMAUG-T) and four digital signature schemes (AIMer, HAETAE, MQ-Sign, NCC-Sign). On January 16, 2025, the final algorithms for the KpqC standard were announced. The selected Key Encapsulation Mechanisms (KEM/PKE) are NTRU+ and SMAUG-T, while AIMer and HAETAE were chosen as the standard digital signature algorithms. The parameter sizes for these selected algorithms are summarized in Table 2.

Table 2: Parameter Sizes of KpqC Candidate Algorithms

Type	**Algorithm**	**Security Level**	**Public Key (Bytes)**	**Secret Key (Bytes)**	**Ciphertext/Sig. (Bytes)**
Key Encapsulation Mechanisms (KEM)					
SMAUG-T	smaug_t1	1	672	832	672
	smaug_t3	3	1,088	1,312	992
	smaug_t5	5	1,440	1,728	1,376
NTRU+	ntru_plus_kem576	1	864	1,760	864
	ntru_plus_kem768	3	1,152	2,336	1,152
	ntru_plus_kem864	3	1,296	2,624	1,296
	ntru_plus_kem1152	5	1,728	3,488	1,728
Digital Signature Algorithms (DSA)					
HAETAE	haetae_2	2	992	1,408	1,474
	haetae_3	3	1,472	2,112	2,349
	haetae_5	5	2,080	2,752	2,948
AIMer	aimer_128s	1	32	48	4,160
	aimer_128f	1	32	48	5,888
	aimer_192s	3	48	72	9,120
	aimer_192f	3	48	72	13,056
	aimer_256s	5	64	96	17,056
	aimer_256f	5	64	96	25,120

2.5 Related Work

Previous research on post-quantum cryptography has followed three main trajectories, yet none have addressed the practical performance of KpqC in real-world TLS environments.

First, while several studies have benchmarked KpqC algorithms at the implementation level, they measure performance in isolation, overlooking the overhead of protocol integration and network communication [18, 19]. Second, extensive research on integrating PQC into TLS has focused exclusively on NIST standards, leaving a critical gap in the evaluation of national standards like KpqC [8, 9]. Finally, studies on hybrid certificate frameworks have remained largely theoretical, proposing structures without implementing or measuring them with non-NIST algorithms in production-like environments [6, 7].

Our work directly addresses these gaps by providing the first comprehensive performance evaluation of KpqC-based hybrid certificates within a production-ready TLS 1.3 implementation, offering crucial data for real-world deployment decisions.

3 Proposed Method

Our research aims to provide a quantitative analysis of the static (certificate size) and dynamic (handshake performance) overheads associated with the PQC transition. To address this gap, our methodology first involved integrating the KpqC algorithms into the OQS ecosystem, thereby establishing a standardized testbed. Subsequently, we conducted two primary experiments using this framework: certificate size measurement and TLS handshake performance benchmarking. In this section, we describe step-by-step (i) the porting of the KpqC algorithm, and (ii) the static and dynamic overhead measurement procedures. The entire workflow is summarized in Figure 1.

3.1 Porting and Implementation of KpqC Algorithms in the OQS Framework

A key contribution of this work was to enable the first empirical evaluation of KpqC candidates in a standardized environment, as they were not officially supported by the OQS framework at the time of this research. To facilitate a fair and reproducible comparison against NIST PQC standards, we ported major KpqC signature (HAETAE, AIMer) and KEM (SMAUG-T, NTRU+) algorithms into the `liboqs` library and extended the corresponding `oqs-provider` for OpenSSL 3.x.

This integration process was guided by a systematic methodology focused on ensuring compatibility, modularity, and correctness. For standardized API compliance, we implemented API wrappers to ensure that the original KpqC source code interfaces seamlessly with the standardized API defined by `liboqs` (e.g., `OQS_SIG_keypair`, `OQS_SIG_sign`), providing a consistent access method for

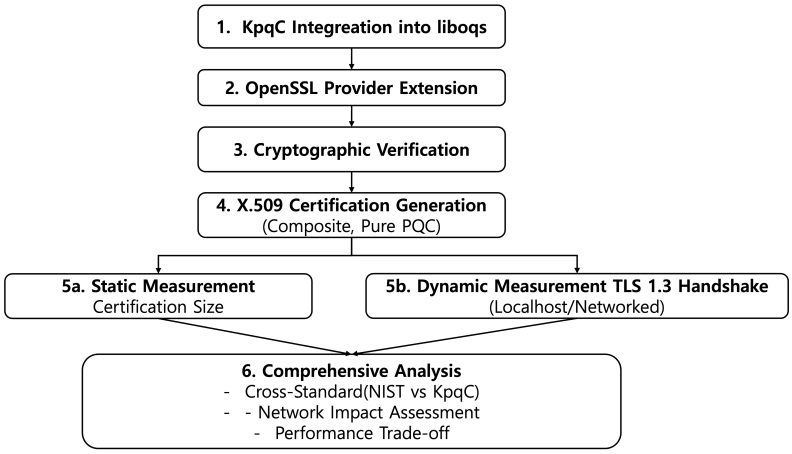

Fig. 1: Methodology Workflow for Evaluating the Practical Overhead of KpqC in X.509 Certificates and TLS 1.3 Handshakes

any high-level application, such as the `oqs-provider`. For build system integration and modularity, each KpqC algorithm was integrated as a distinct module within the CMake-based build system of `liboqs`. This allows users to selectively compile only the necessary algorithms via conditional compilation flags (e.g., `OQS_ENABLE_SIG_haetae_2`), enhancing library optimization and extensibility. Finally, for validation and verification, we performed rigorous verification using the Known Answer Test (KAT) vectors provided by the official KpqC implementations. These tests, integrated into the `liboqs` testing framework, confirmed that our implementation produces cryptographically identical results to the original reference code.

3.2 Measurement Procedure

Static Overhead: Certificate Size Measurement To quantify the static overhead, we generated X.509 certificates for a wide range of signature schemes, including classical ECC, PQC-only, and composite hybrid certificates. The physical size of each certificate was measured by extracting its ASN.1 DER-encoded byte stream using the following command:

```
openssl x509 -in [cert.pem]-outform DER | wc -c
```

Dynamic Overhead: TLS Handshake Performance Measurement. To evaluate the dynamic overhead, we benchmarked the TLS 1.3 handshake performance by varying the key exchange mechanism (KEM) negotiated between the client and server. For these tests, the server was configured with a standard ECC (`secp256r1`) certificate to isolate the performance impact of the KEM computation. Each benchmark consisted of 200 consecutive handshakes, from which

the average time was calculated. The performance was measured in two distinct scenarios:

- **Scenario 1 (Computational Performance):** To isolate pure computational overhead without network latency, both the client and server were executed on a high-performance workstation (MacBook Pro) communicating via the `localhost` interface.
- **Scenario 2 (Networked and Resource-Constrained Performance):** To analyze the impact of a realistic network environment and resource constraints, the server was executed on an embedded device (Raspberry Pi 5), with the client connecting from the workstation over a local network.
  The Raspberry Pi 5 was chosen because its ARM-based architecture and limited resources make it a suitable representative model for modern high-performance IoT gateways or edge computing devices.

4 Results and Analysis

To quantitatively assess the impact of integrating post-quantum cryptography into existing infrastructures, we performed two core experiments. The first focused on measuring the increase in certificate sizes, while the second examined the effect on TLS 1.3 handshake performance.

4.1 Experimental Setup

The evaluation used two setups. In the computational scenario, client and server ran on a single MacBook Pro (M1 Pro, 16 GB RAM) to isolate crypto overhead. In the networked scenario, a MacBook client communicated over IEEE 802.11ac with a Raspberry Pi 5 server (ARM-A76, 8 GB RAM) at a static IP. Both systems used OpenSSL 3.1.2 with a custom OQS provider.

4.2 Static Overhead: Certificate Size Analysis

Various X.509 certificates were generated based on the signature schemes described in the methodology. The size of each certificate was measured in terms of its DER-encoded length. Results confirmed significant growth in certificate size when transitioning from classical cryptographic schemes to PQC-based alternatives. For example, at NIST Security Level 1, a classical secp256r1 certificate measured 385 bytes, whereas the smallest PQC alternative, Falcon512, measured 1,788 bytes, representing an approximately 4.6-fold increase. KPQC algorithms exhibited even greater sizes, ranging between 4,427 bytes (aimer128s) and 6,155 bytes (aimer128f), corresponding to increases of 11.5× to 16× compared to classical counterparts.

In addition, analysis of hybrid certificates showed that the size increase attributable to hybridization was consistently modest across all tested algorithms.

Table 3: Comparison of DER-Encoded Certificate Sizes for PQC and Hybrid Signature Schemes

Family	Algorithm	Level	PQC-Only (B)	Hybrid (B)
NIST Security Level 1 & 2				
Classical	secp256r1	1	385	-
NIST PQC	falcon512	1	1788	1941
KpqC	aimer128s	1	4427	4582
KpqC	aimer128f	1	6155	6310
NIST PQC	sphincsshake128fsimple	1	17,382	17,536
NIST PQC	mldsa44	2	3977	4120
KpqC	haetae2	2	2702	2857
NIST Security Level 3				
Classical	secp384r1	3	447	-
NIST PQC	mldsa65	3	5506	5713
KpqC	haetae3	3	4057	4276
KpqC	aimer192s	3	9404	9624
KpqC	aimer192f	3	13,340	13,559
NIST Security Level 5				
Classical	secp521r1	5	521	-
NIST PQC	mldsa87	5	7464	7742
NIST PQC	falcon1024	5	3304	3596
KpqC	haetae5	5	5264	5554
KpqC	aimer256s	5	17,356	17,647
KpqC	aimer256f	5	25,420	25,711

This indicates that the PQC component itself constitutes the primary contributor to certificate size, while the addition of classical signatures has minimal impact.

We generated a comprehensive set of X.509 certificates using the signature schemes detailed in the methodology. The real-world size of each certificate was measured by extracting its DER-encoded byte stream. The complete measurement results are presented in Table 3, organized by NIST security level to facilitate fair comparison. Figure 2 visualize these results for Level 1 and 2 algorithms.

Overall Size Comparison Certificate sizes increase markedly across security levels: at SL1, secp256r1 certificates grow from 385 B to 1788 B (Falcon512, 4.6×) and 4427–6155 B for KpqC (11.5–16×); at SL3, secp384r1's 447 B rises to 5506 B (ML-DSA-65, 12.3×) and 4057–13340 B for KpqC (9.1–29.8×); and at SL5, secp521r1's 521 B expands up to 25420 B (AIMer-256f, 48.8×).

Hybrid vs. PQC-Only Overhead Analysis shows hybrid certificates add minimal overhead (2–6%) over PQC-only variants—for example, p256_haetae2

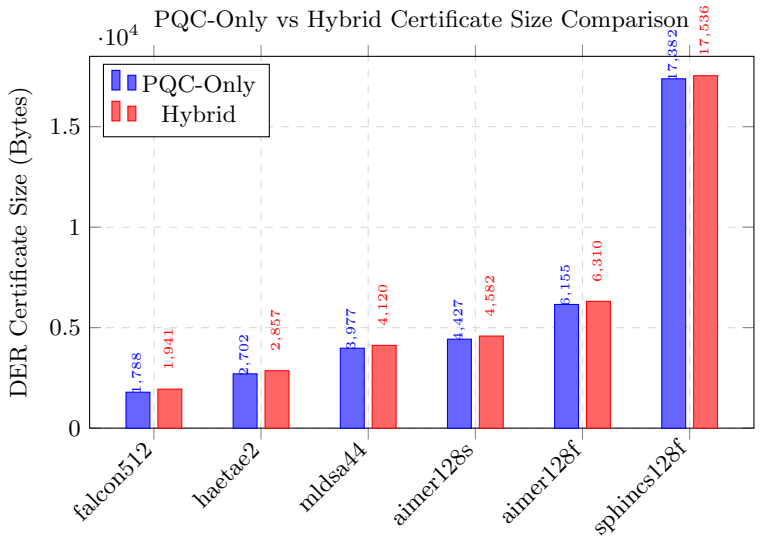

Fig. 2: Direct comparison of PQC-only and hybrid certificates for NIST security levels 1 and 2. Hybridization adds only modest overhead. Note: "sphincs128f" denotes "sphincsshake128fsimple."

is only 155 B (+5.7%) larger than haetae2, p256_mldsa44 adds 143 B (+3.6%), and p384_aimer192s adds 220 B (+2.3%). This confirms that PQC signatures dominate certificate size, with classical signatures contributing only marginally.

Algorithm Family Comparison At equivalent security levels, NIST's Falcon and ML-DSA produce smaller certificates than KpqC algorithms. For example, HAETAE2 yields 2,702 bytes versus ML-DSA44's 3,977 bytes. Conversely, AIMer's fast variants (aimer128f, aimer192f, aimer256f) generate certificates approximately 40–48% larger than their compact counterparts.

This variation is presumed to stem from the differing mathematical structures underlying each family, such as the Module-LWE problem in ML-DSA and HAETAE versus the MPC-in-the-head paradigm for AIMer, as well as the efficiency of their respective proof techniques like Fiat-Shamir with Aborts.

4.3 Dynamic Overhead: Handshake Performance Analysis

We conducted TLS 1.3 handshake performance measurements using standard ECC certificates on the server while varying the key exchange mechanism. Tests were performed in two scenarios: localhost (computational overhead isolation) and networked (realistic deployment simulation).

Table 4: Performance of PQC Hybrid Key Exchange Schemes in TLS 1.3 Handshakes (200 connections on Macbook-localhost)

Family	Key Exchange Scheme	Time (ms)	Throughput (hps)
NIST Security Level 1			
Classical	secp256r1 (Baseline)	5.24	190.70
NIST PQC	secp256r1 + mlkem512	5.20	192.32
KpqC	secp256r1 + smaug_t1	45.00	22.22
KpqC	secp256r1 + ntru_plus_kem576	45.09	22.18
NIST Security Level 3			
Classical	secp384r1 (Baseline)	5.19	192.51
NIST PQC	secp384r1 + mlkem768	5.23	191.27
KpqC	secp384r1 + smaug_t3	46.54	21.49
KpqC	secp384r1 + ntru_plus_kem768	46.06	21.71
KpqC	secp384r1 + ntru_plus_kem864	46.19	21.65
NIST Security Level 5			
Classical	secp521r1 (Baseline)	5.18	193.11
NIST PQC	secp521r1 + mlkem1024	5.19	192.79
KpqC	secp521r1 + smaug_t5	47.15	21.21
KpqC	secp521r1 + ntru_plus_kem1152	46.69	21.42

Computational Performance (Localhost - MacBook to MacBook) The localhost computational performance results, presented in Table 4, demonstrate remarkable efficiency for NIST ML-KEM implementations. At Security Level 1, ML-KEM requires 5.20ms compared to the 5.24ms baseline, representing only a +0.8% increase. Security Level 3 shows similarly minimal impact with 5.23ms versus the 5.19ms baseline (+0.8%), while Security Level 5 achieves 5.19ms compared to 5.18ms baseline (+0.2%). These results indicate that NIST ML-KEM adds essentially negligible computational overhead across all security levels.

In stark contrast, KpqC algorithms demonstrate substantial performance penalties in pure computational scenarios. SMAUG-T requires 45.00-47.15ms across security levels, representing 8.6-9.1× slower performance than the baseline. NTRU+ shows comparable overhead with 45.09-46.69ms across security levels, translating to 8.5-9.0× slower performance than baseline measurements. The performance disparity is dramatic: while NIST ML-KEM adds negligible computational overhead (<1%), KpqC candidates increase handshake time by approximately 850-900% in pure computational scenarios.

Networked Performance (MacBook Client to Raspberry Pi Server) The networked performance results, shown in Table 5, reveal how network latency and resource constraints significantly impact baseline performance. Network communication combined with weaker server hardware increases baseline handshake times substantially across all security levels. Level 1 performance de-

Table 5: Performance of Hybrid Key Exchange Schemes in TLS 1.3 Handshakes (200 connections on MacBook-Raspberry Pi)

Family	Key Exchange Scheme	Time (ms)	Overhead vs. ECC
NIST Security Level 1			
Classical	secp256r1 (Baseline)	98.32	-
NIST PQC	secp256r1 + mlkem512	130.14	+32.36 %
KpqC	secp256r1 + smaug_t1	130.15	+32.37 %
KpqC	secp256r1 + ntru_plus_kem576	135.68	+38.00 %
NIST Security Level 3			
Classical	secp384r1 (Baseline)	109.90	-
NIST PQC	secp384r1 + mlkem768	144.75	+31.71 %
KpqC	secp384r1 + smaug_t3	152.04	+38.34 %
KpqC	secp384r1 + ntru_plus_kem768	146.70	+33.48 %
KpqC	secp384r1 + ntru_plus_kem864	150.63	+37.06 %
NIST Security Level 5			
Classical	secp521r1 (Baseline)	125.19	-
NIST PQC	secp521r1 + mlkem1024	167.41	+33.72 %
KpqC	secp521r1 + smaug_t5	176.40	+40.90 %
KpqC	secp521r1 + ntru_plus_kem1152	166.04	+32.63 %

grades from 5.24ms to 98.32ms (18.8× increase), Level 3 expands from 5.19ms to 109.90ms (21.2× increase), and Level 5 grows from 5.18ms to 125.19ms (24.2× increase).

Despite the increased baseline times, relative overhead patterns shift dramatically in networked environments. NIST ML-KEM demonstrates +31.7-33.7% overhead over respective baselines, while KpqC SMAUG-T shows +32.4-40.9% overhead and KpqC NTRU+ exhibits +32.6-38.0% overhead over baseline measurements. While these relative increases appear more manageable than the localhost scenario, absolute performance remains concerning. ML-KEM achieves 130-167ms (2.5-3.2× slower than localhost), while KpqC algorithms require 130-176ms (2.9-3.7× slower than localhost).

Performance Analysis In networked environments, the relative handshake overhead for KpqC algorithms falls from approximately 850% (localhost) to around 30–40%, indicating that network latency and server limitations dominate total response time. However, absolute KpqC handshake durations remain 30–45 ms longer than NIST PQC baselines, which can be prohibitive for interactive applications targeting sub-100 ms responses. These results show that, although relative overhead is substantially reduced over the network, the absolute latency penalty still demands careful consideration in real-world deployments.

This suggests that in IoT or mobile settings, where network delays are the main bottleneck, KpqC's extra computation may be acceptable, and pure CPU-only benchmarks can overstate real-world barriers.

4.4 Discussion and Key Findings

Performance Trade-offs Performance results highlight differences in algorithm maturity. NIST ML-KEM exhibits optimized performance with minimal overhead, while KpqC candidates incur significant penalties, reflecting either inherent algorithmic complexity or current implementation limitations that could improve with further development. Our integrated analysis highlights a tension between optimal static and dynamic choices: Falcon-512 yields the smallest certificates but, being signature-only, cannot be evaluated for handshake latency, whereas ML-KEM offers near-zero latency yet pairs with ML-DSA, which produces certificates over twice the size of Falcon-512 at the same security level. This mismatch underscores the necessity of jointly assessing both size and performance when selecting PQC components.

Certificate Storage and Transmission Implications Our measurements show substantial increases in certificate size, posing practical challenges. Enterprise PKI systems may face significant storage burdens, while mobile and IoT devices could experience higher latency during certificate validation due to bandwidth constraints. Larger certificates may also reduce caching efficiency, affecting system performance.

Deployment Considerations The networked performance results offer key insights for deployment planning. While high-latency networks may mask PQC overhead, local and edge environments will face the full computational cost, demanding careful hardware planning. For KpqC algorithms, an 8-9× processing increase may require substantial infrastructure upgrades in high-throughput scenarios.

Limitations This study has several limitations. Results are based on specific implementations and hardware, and may differ on systems with cryptographic acceleration. Network tests were limited to LAN conditions, not covering broader internet variability. Additionally, certificate size analysis focused solely on DER encoding, without exploring potential compression techniques.

5 Conclusion

This paper quantified static and dynamic overheads of PQC in X.509/TLS for KpqC versus NIST PQC: certificate sizes grow 4.6–48× (Falcon–AIMer), and

handshake latency is <1% for ML-KEM but >750% for KpqC in pure computation, with the gap narrowing under network constraints. These results underscore the bandwidth/storage versus compute-latency trade-off, guiding designers in PQC algorithm selection. Future work should evaluate other hybrid frameworks (e.g., chameleon) by measuring both static size and dynamic metrics such as path validation time. Extending benchmarks to realistic WAN environments with variable latency and packet loss will further clarify real-world deployment challenges.

References

1. "NIST PQC project." `https://csrc.nist.gov/Projects/post-quantum-cryptography`. Accessed: 2025-06-07.
2. R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber algorithm specifications and supporting documentation," *NIST PQC Round*, vol. 2, no. 4, 2019.
3. L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-Dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 238–268, 2018.
4. P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-fourier lattice-based compact signatures over NTRU," *Submission to the NIST's post-quantum cryptography standardization process*, vol. 36, no. 5, 2018.
5. D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ signature framework," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pp. 2129–2146, 2019.
6. A. C. Chen, "Post-quantum cryptography x. 509 certificate," in *2024 International Conference on Smart Systems for applications in Electrical Sciences (ICSSES)*, pp. 1–6, IEEE, 2024.
7. P. Kampanakis, P. Panburana, E. Daw, and D. Van Geest, "The viability of post-quantum x. 509 certificates," *Cryptology ePrint Archive*, 2018.
8. M. A. Garrach, C. Waghela, M. M. Mathews, and L. S. Sreekuttan, "Benchmarking speed of post-quantum lattice based pke/kem schemes using liboqs," in *2022 International Conference on Trends in Quantum Computing and Emerging Business Technologies (TQCEBT)*, pp. 1–5, IEEE, Oct. 2022.
9. F. Opiłka, M. Niemiec, M. Gagliardi, and M. A. Kourtis, "Performance analysis of post-quantum cryptography algorithms for digital signature," *Applied Sciences*, vol. 14, no. 12, p. 4994, 2024.
10. "Open Quantum Safe." `https://openquantumsafe.org/`. Accessed: 2025-06-07.
11. "liboqs." `https://github.com/open-quantum-safe/liboqs`. Accessed: 2025-06-07.
12. M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X. 509 internet public key infrastructure online certificate status protocol-ocsp," tech. rep., 1999.
13. "Migration to post-quantum cryptography quantum readiness: Testing draft standards." `https://www.nccoe.nist.gov/sites/default/files/2023-12/pqc-migration-nist-sp-1800-38c-preliminary-draft.pdf`. Accessed: 2025-06-07.

14. "pq-hybrid-x509." https://datatracker.ietf.org/doc/draft-truskovsky-lamps-pq-hybrid-x509/. Accessed: 2025-06-07.
15. "IETF chameleon certificates." https://datatracker.ietf.org/doc/draft-bonnell-lamps-chameleon-certs/. Accessed: 2025-06-07.
16. "IETF composite certificates." https://datatracker.ietf.org/doc/draft-ounsworth-pq-composite-sigs/09/. Accessed: 2025-06-07.
17. Y. Choi, M. Kim, Y. Kim, J. Song, J. Jin, H. Kim, and S. C. Seo, "KpqBench: Performance and implementation security analysis of KpqC competition round 1 candidates," *IEEE Access*, 2024.
18. M. Sim, S. Eum, G. Song, M. Lee, S. Kim, M. Song, and H. Seo, "KpqClean ver2: Comprehensive benchmarking and analysis of KpqC algorithm round 2 submissions," *Cryptology ePrint Archive*, 2024.
19. H. Kwon, M. Sim, G. Song, M. Lee, and H. Seo, "Evaluating KpqC algorithm submissions: Balanced and clean benchmarking approach," in *International Conference on Information Security Applications*, pp. 338–348, Springer, 2023.